Ver el fichero adjunto para información sobre las funciones.

Objetivo de la práctica

Crear un programa servidor y un programa cliente en TCP y entender su funcionamiento.

1. Cliente/servidor básicos. Sigue los pasos del documento adjunto para crear una conexión TCP cliente/servidor en el que el servidor envíe un mensaje de saludo al cliente.
    a. Programa servidor TCP (`servidor.c`):
        - Debe escuchar por todas sus interfaces usando un número de puerto indicado como parámetro en línea de comandos
        - Debe poder atender múltiples conexiones secuenciales de clientes
        - Para cada cliente que se conecte, debe imprimir en pantalla la IP y el puerto del cliente y enviarle un mensaje de saludo (una cadena de caracteres ASCII)
    b. Programa cliente TCP (`cliente.c`):
        - Debe permitir indicar la IP y el puerto del servidor como parámetros en línea de comandos (primero la IP y después el puerto)
        - Se debe conectar al servidor y recibir el mensaje del mismo
        - Debe imprimir el mensaje recibido y el número de bytes que se han recibido
    c. Comprueba si es posible que el servidor envíe dos mensajes con sendas funciones send() y que el cliente reciba ambos mensajes con una única sentencia recv(). Haz que el cliente espere un breve tiempo entre que establece la conexión y recibe el mensaje, para que al servidor le dé tiempo de enviar ambos mensajes (puedes usar la función sleep para la espera). Describe las modificaciones introducidas en los códigos y el resultado obtenido (número de bytes recibidos y el mensaje). Comprueba qué pasa si eliminas la espera.
    d. En el caso anterior, elimina la espera y prueba a usar en el cliente un bucle de tipo `while( (n=recv(socket,mensaje,tamano_mensaje,0)) > 0 ){...}`. Probar a especificar varios valores en el número de bytes a recibir (por ejmplo, 5, 10...). Describe las modificaciones introducidas en los códigos y el resultado obtenido. ¿Cuándo se termina el bucle?
2. Servidor de mayúsculas. Hacer un programa servidor (`servidormay.c`) y un programa cliente (`clientemay.c`) en el que el cliente lea un archivo de texto de entrada (pasándole el nombre en línea de comandos, antes de la IP y el puerto), y se lo envíe línea a línea al servidor usando la misma conexión. El servidor debe pasar los caracteres de cada línea a mayúsculas (podéis usar la función `toupper()` para hacer esa conversión) y devolverle al cliente la línea convertida. Por último, el cliente va recibiendo las líneas y las va escribiendo en un archivo de salida, que debe tener el mismo nombre que el archivo de entrada pero todo en mayúsculas (nombre y extensión). El funcionamiento debe ser el siguiente: una vez establecida la conexión
    a. el cliente lee una línea del archivo de entrada y se la envía al servidor
    b. el servidor se la devuelve al cliente pasada a mayúsculas
    c. el cliente la escribe en el archivo de salida
    d. de vuelta al paso a, hasta que se termine el archivo.

    Además, el servidor debe aceptar como parámetro de entrada el número de puerto por el que atiende las solicitudes y el cliente el nombre del fichero de entrada, la IP y puerto del servidor, por ese orden. Del mismo modo que en el ejercicio 1, el servidor debe imprimir en pantalla la IP y el puerto del cliente.
3. Comprobación de que el servidor del punto 2 puede atender varios clientes secuencialmente. En el lazo del cliente donde se van leyendo las líneas del archivo, introducir un sleep para que dé tiempo a lanzar un segundo cliente en otra terminal. Describe lo que ocurre.

Requisitos mínimos

1. Se debe hacer uso de las funciones vistas en la clase.
2. Se debe tener en cuenta que las líneas pueden ser muy largas (máximo de 1000 caracteres).
3. Toda llamada a una función del sistema debe tener su correspondiente chequeo de error.
4. Toda función debe salir con el mensaje de error y el código apropiado en caso de error.
5. El código no puede fallar aunque se usen datos de entrada incorrectos.
6. Toda memoria reservada dinámicamente debe liberarse correctamente.
7. El código debe de estar **adecuadamente** comentado, indicándose de forma clara qué se hace en todas las funciones definidas, junto con la explicación de los parámetros de entrada y salida de las mismas.
8. El código deberá estar correctamente formateado y tabulado.
9. El código debe compilar en un sistema Linux con gcc. Si se usa Windows, se debe convertir el fichero con el código a formato Unix. Los usuarios de Mac debéis tener cuidado porque hay diferencias con Linux.
10. Los programas deben funcionar aunque el cliente y el servidor se ejecuten en ordenadores diferentes.
11. Los parámetros de entrada necesarios deben ser proporcionados como argumentos del main, sin usar la función `getopt()`.
12. Se penalizará que aparezcan mensajes de Warning en la compilación (con la opción *-Wall*)

?

Entrega

Enviar un único archivo comprimido con zip, que contenga:

- Los códigos (no los ejecutables) de los puntos 1.a (servidor básico, `servidor.c`), 1.b (cliente básico, `cliente.c`) y 2 (servidor y cliente de mayúsculas, `servidormay.c` y `clientemay.c`, respectivamente).
- Informe sobre los puntos 1.c, 1.d y 3.
- Una única captura de pantalla en la que se pueda ver que el cliente y servidor de mayúsculas se están ejecutando en distintos ordenadores. NOTA: no es posible conectar dos PCs usando la Eduroam (temas de cortafuegos), para hacer esta prueba podéis usar dos PCs del aula conectándoos de uno al otro mediante SSH (desde un terminal hacéis: ssh nombre_usuario@IP_otroPC), o usando dos máquinas virtuales en vuestro PC.

P2-P3_funciones.pdf

Editar entrega | Retirar a entrega

Estado da entrega

Estado da entrega	
Estado das cualificacións	
Tempo restante	
Última modificación	
Entregas de ficheiros	
Comentarios a entrega	